

Reviewer Pack — Real Rank of Communication Matrix Bounds AC^0 Circuit Size fo...

Entry 9bf36053ab4b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 22:18:16 UTC

Real Rank of Communication Matrix Bounds AC^0 Circuit Size for PARITY

Entry ID: 9bf36053ab4b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 22:18:16 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC^0 Circuit Size for PARITY

Statement:

For any AC^0 circuit C computing PARITY on n bits, the real rank of its communication matrix is $\Omega(\log n)$.

Rationale (proposer's reasoning):

Real rank captures the minimal number of rank-1 matrices needed to express the communication matrix, reflecting the function's geometric complexity. High real rank implies the function cannot be compressed into small AC^0 circuits, aligning with known lower bounds for PARITY.

Taxonomy category: AC^0_PARITY (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 72485c6fab04d87c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=1.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    depth = 3
    size_limit = 2**10

    # Generate a random AC0 circuit computing PARITY on n bits
    def generate_ac0_circuit(n, depth, size_limit):
        if depth == 0:
            return [random.choice([0, 1])]
        else:
            inputs = generate_ac0_circuit(n, depth - 1, size_limit)
            outputs = []
            for i in range(size_limit):
                gate_type = random.choice(['OR', 'AND'])
                if gate_type == 'OR':
                    output = any(inputs[j] for j in range(len(inputs)))
                else:
                    output = all(inputs[j] for j in range(len(inputs)))
                outputs.append(output)
            return outputs

    circuit = generate_ac0_circuit(n, depth, size_limit)

    # Compute the communication matrix
    def compute_communication_matrix(circuit):
        m = len(circuit)
        n = int(math.log2(m))
        comm_matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(m):
            x = bin(i)[2:].zfill(n)
            y = bin(circuit[i])[2:].zfill(n)
            for j in range(n):
                if x[j] == '1':
                    comm_matrix[j][n] += 1
                    comm_matrix[n][j] -= 1
        return comm_matrix
```

```

comm_matrix = compute_communication_matrix(circuit)

# Calculate the real rank via Gaussian elimination over R
def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    for i in range(m):
        if matrix[i][i] == 0:
            return None # Singular matrix
        for j in range(i + 1, m):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

real_rank = gaussian_elimination(comm_matrix)

# Verify the conjecture
if real_rank is None:
    return {
        "metric_name": "real_rank",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "singular_matrix"
    }

metric_value = real_rank
conjecture_holds = real_rank >= 0.1 * math.log(n)
counterexample = "" if conjecture_holds else f"real_rank={real_rank}, expected ≥ {0.1 * math.log(n)}

return {
    "metric_name": "real_rank",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r["metric_value"] is not None) /
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    for result in results:
        if not result["conjecture_holds"]:
            print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
            break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

se, 'counterexample': 'singular_matrix'}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'real_rank', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True}
RESULT: FALSIFIED counterexample="singular_matrix" first_failing_seed=929

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

Metric definition bug: the test flags any ‘singular_matrix’ as a counterexample, but the conjecture only requires rank $\Omega(\log n)$, not full rank. A singular matrix can easily have rank far exceeding $\log n$ (e.g., an $n \times n$ matrix of rank $n-1$ is singular yet has rank $\approx n$). Additionally metric_value is None on every trial, meaning the actual real rank was never computed — the FALSIFIED verdict is based on a proxy (non-invertibility) that has no logical relationship to the $\Omega(\log n)$ lower bound being

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The FALSIFIED result rests on a buggy metric: ‘singular_matrix’ is not a valid counterexample to an $\Omega(\log n)$ rank lower bound (singular matrices can have rank $\approx \log n$), and metric_value was None on every trial, so the actual real rank was never measured. The critic’s CHALLENGE is well-founded and the pre-registered criterion cannot be evaluated. | next: Fix the test to compute `numpy.linalg.matrix_rank` of a communication matrix actually derived from an AC^0 PARITY circuit, then compare it against

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	105062
2	preregistration	ollama_remote	qwen3:8b	0	0	34253
3	novelty	ollama_remote	qwen3:8b	0	0	28178
4	novelty	ollama_remote	qwen3:8b	0	0	20207
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11549
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11322
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8592
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11149
9	critic	claude_max	opus	0	0	17593
10	judge	claude_max	opus	0	0	5687

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 253592 ms total latency. Provider mix: {'ollama_remote': 8, 'claude_max': 2}

(full prompt+response transcripts available in `research/audit/9bf36053ab4b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9bf36053ab4b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9bf36053ab4b.tar.gz` (if generated)